



RECURRENT MARKED TEMPORAL POINT PROCESSES: EMBEDDING EVENT HISTORY TO VECTOR

Presented by Josh Job Joseph



INTRODUCTION

“Based on the observed sequence of events, can we predict what kind of event will take place at what time in the future?”

Current studies go about this in two different manners:

Varying-Order Markov Model

AND

Marked Temporal Point Processes and Intensity Functions





MARKOV MODELS

Based on the observed sequence of states, the model can predict the most likely state the process will evolve into on the next step.

This gives the model a couple of issues.

One, it assumes that the process proceeds by unit time-steps (exclusively)

Two, when the number of states is large, then the Markov model cannot capture long term dependencies due to the state-space growing exponentially for the number of steps being considered.





MARKED TPP

Marked TPP and intensity functions are a more general mathematical framework to capture these relationships.

Common issues with typical point processes and intensity functions, such as Hawkes and autoregressive conditional duration processes, make specific assumptions about the functional forms of the generative process.





Recurrent Marked Temporal Point Process

Their idea is to view the intensity function of a temporal point process as a nonlinear function of the history of the process and parameterize the function of the history of the process, and parameterize the function using a recurrent neural network.



Recurrent Marked Temporal Point Process

Event history is embedded into a compact vector representation which can be used for predicting the next event time and marker type.

PARAMETERIZATION

Within a small window $[t, t + dt)$, $\lambda^*(t)dt$ is the probability for the occurrence of a new event given the history H_t :

$$\lambda^*(t)dt = P \{ \text{event in } [t, t + dt) | H_t \}$$

Given the conditional density function $f^*(t)$, the conditional intensity function can be specified as:

$$\lambda^*(t)dt = \frac{f^*(t)dt}{S^*(t)} = \frac{f^*(t)dt}{1 - F^*(t)}$$

$F^*(t)$ is the cumulative probability that a new event will happen before time t since the last event time t_n , and $S^*(t)$ is the respective probability that no new event has ever happened up to time t since t_n .

$$S^*(t) = \exp \left(- \int_{t_n}^t \lambda^*(\tau) d\tau \right)$$

$$f^*(t) = \lambda^*(t) \exp \left(- \int_{t_n}^t \lambda^*(\tau) d\tau \right).$$

POISSON PROCESS

The inter-event times are independent and identically distributed random variables conforming to the exponential distribution. The conditional intensity function is assumed to be independent of the history H_t and keeping constant over time

$$\lambda^*(t) = \lambda_0 \geq 0.$$

HAWKES PROCESS

Hawkes process captures the mutual excitation phenomenon among events with the conditional intensity being defined as

$$\lambda^*(t) = \gamma_0 + \alpha \sum_{t_j < t} \gamma(t, t_j)$$

where $\gamma(t, t_j) > 0$ is the triggering kernel capturing temporal dependencies, $\gamma_0 > 0$ is a baseline intensity independent of the history and the summation of kernel terms is history dependent and a stochastic process by itself. The kernel function can be chosen in advance, e.g., $\gamma(t, t_j) = \exp(-\beta(t - t_j))$ or $\gamma(t, t_j) = I[t > t_j]$, or directly learned from data.

SELF CORRECTING PROCESS

The self-correcting process seeks to produce regular point patterns with the conditional intensity function

$$\lambda^*(t) = \exp \left(\mu t - \sum_{t_i < t} \alpha \right)$$

where $\mu > 0$, $\alpha > 0$. The intuition is that while the intensity increases steadily, every time when a new event appears, it is decreased by multiplying a constant $e^{-\alpha} < 1$, so the chance of new points decreases after an event has occurred recently.

AUTOREGRESSIVE CONDITIONAL DURATION PROCESS

An alternative way of conditional intensity parametrization is to capture the dependency between inter-event durations. Let $d_i = t_i - t_{i-1}$. The expectation for d_i is given by $\psi_i = E(d_i | \dots, d_{i-2}, d_{i-1})$. The simplest form assumes that $d_i = \psi_i \varpi_i$ where ϖ_i is independently and identically distributed exponential variables with expectation one.

$$\lambda^*(t) = \psi_{N(t)}^{-1}$$

where $\psi_i = \gamma_0 + \sum_{j=0}^m \alpha_j d_{i-j}$ to capture the influences from the most recent m durations, and $N(t)$ is the total number of events up to t .



MAJOR LIMITATIONS



CURSE OF MODEL MISSPECIFICATION

All these different parameterizations of the conditional intensity function seek to capture and represent certain forms of dependency on the history in different ways: Poisson process makes the assumption that the duration is stationary; Hawkes process assumes that the influences from past events are linearly additive towards the current event; the Self-correcting process specifies a non-linear dependency over these past events; and autoregressive conditional duration model imposes a linear structure between successive inter-event durations.

These different parameterizations encode our prior knowledge about the latent dynamics we try to model. In practice, however, the true model is never known.

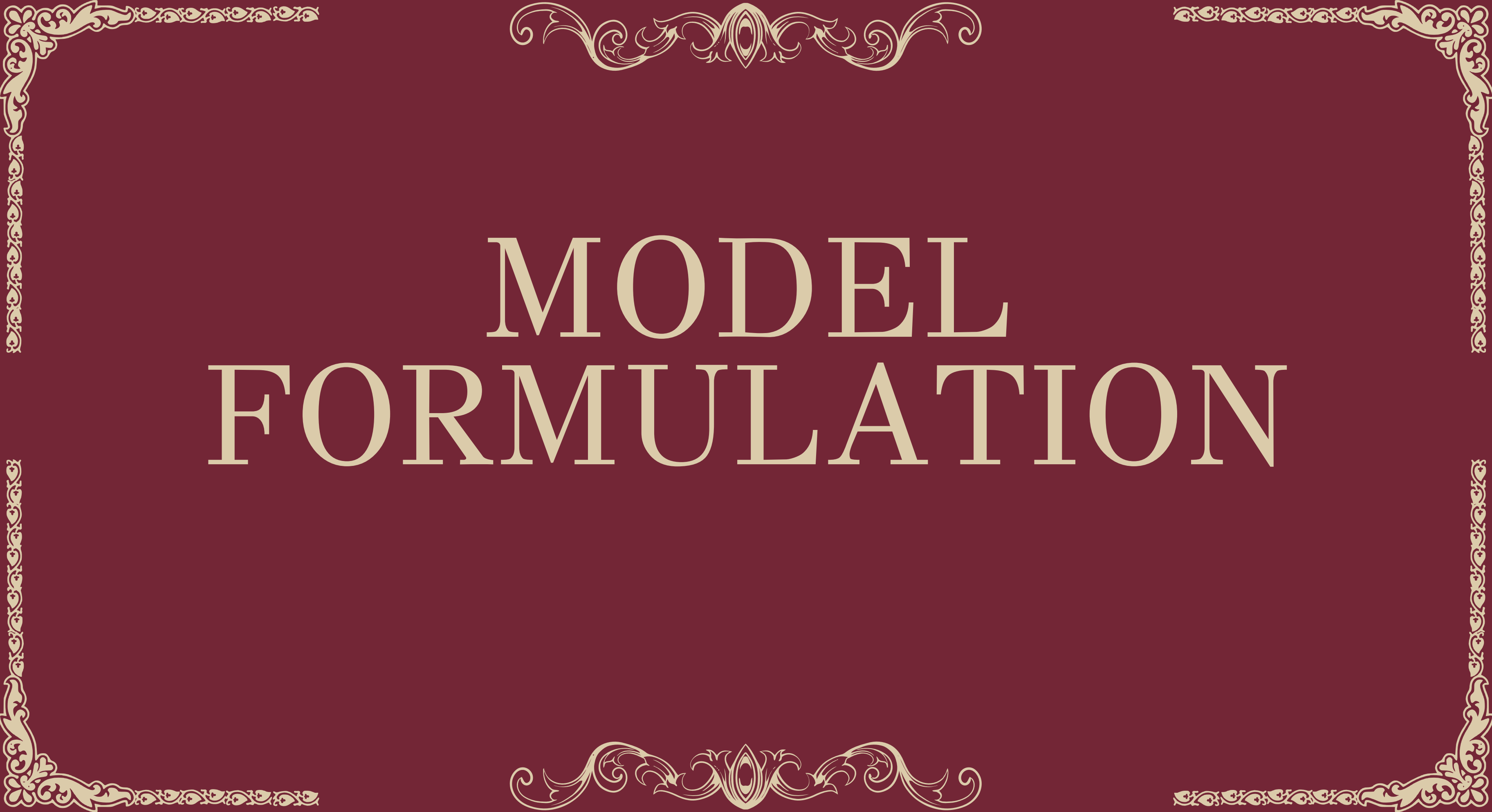


MARKER GENERATION

Classic temporal point processes can be extended to capture the marker information mainly in the following two ways: first, the marker is directly incorporated into the intensity function; second, each marker can be regarded as an independent dimension to have a multi-dimensional temporal point process.

In terms of the former approach, we still need to specify a proper form for the conditional intensity function. Moreover, due to the extra complexity of the function induced by the markers, people normally make strong assumptions that the marker is independent on the history [33], which greatly reduces the flexibility of the model.

With respect to the latter method, it is very often to have large number of markers, which results in a sparsity problem associated with each dimension where only very few events can happen.



MODEL FORMULATION

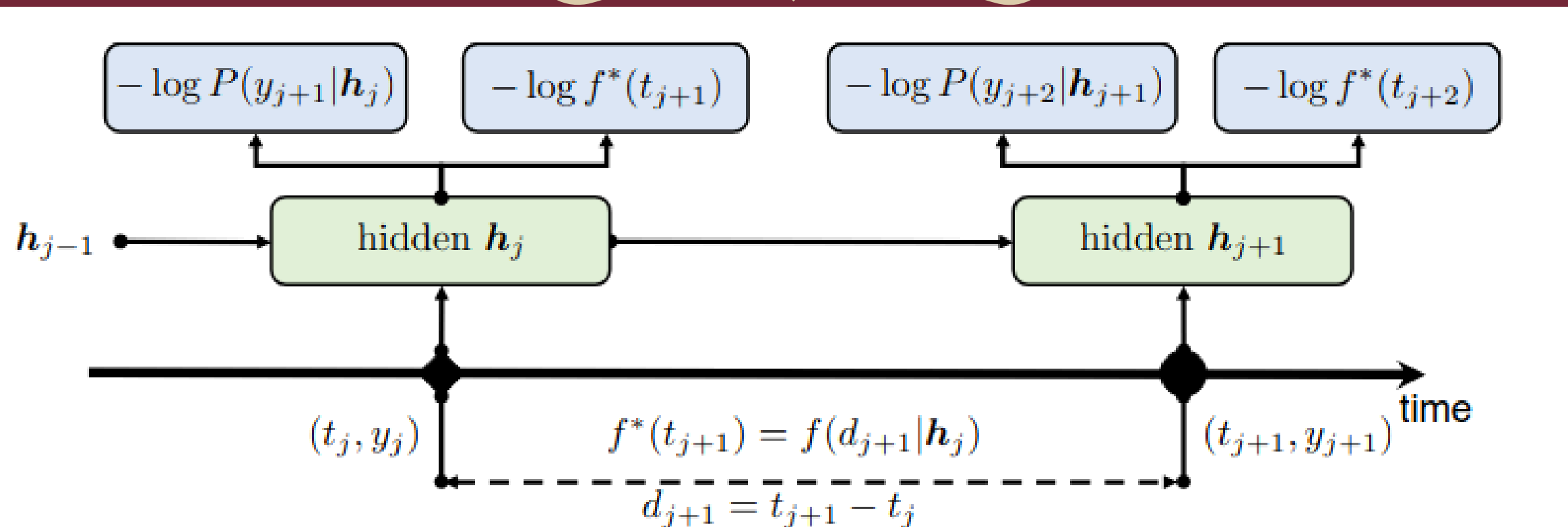


Figure 2: Illustration of Recurrent Marked Temporal Point Process. For each event with the timing t_j and the marker y_j , we treat the pair (t_j, y_j) as the input to a recurrent neural network where the embedding $\mathbf{h}_j$ up to the time t_j learns a general representation of a nonlinear dependency over both the timing and the marker information from past events. Note that the solid diamond and the circle on the timeline indicate two events of different types $y_j \neq y_{j+1}$.

As shown in Figure 2, for the event occurring at the time t_j of type y_j , the pair (t_j, y_j) is fed as the input into a recurrent neural network unfolded up to the $j + 1$ -th event.

The embedding h_{j-1} represents the memory of the influence from the timings and the markers of past events. The neural network updates h_{j-1} to h_j by taking into account the effect of the current event (t_j, y_j) .

Since now h_j represents the influence of the history up to the j -th event, the conditional density for the next event timing can be naturally represented as:

$$f^*(t_{j+1}) = f(t_{j+1} | \mathcal{H}_t) = f(t_{j+1} | \mathbf{h}_j) = f(d_{j+1} | \mathbf{h}_j)$$

where $d_{j+1} = t_{j+1} - t_j$. As a consequence, we can depend on h_j to make predictions to the timing $\hat{t}_{j+1}$ and the type $\hat{y}_{j+1}$ of the next event.



The advantage of this formulation is that we explicitly embed the event history into a latent vector space, and by the elegant relation (4), we are now able to capture a general form of the conditional intensity function $\lambda^*(t)$ without the need of specifying a fixed parametric specification for the dependency structure over the history.

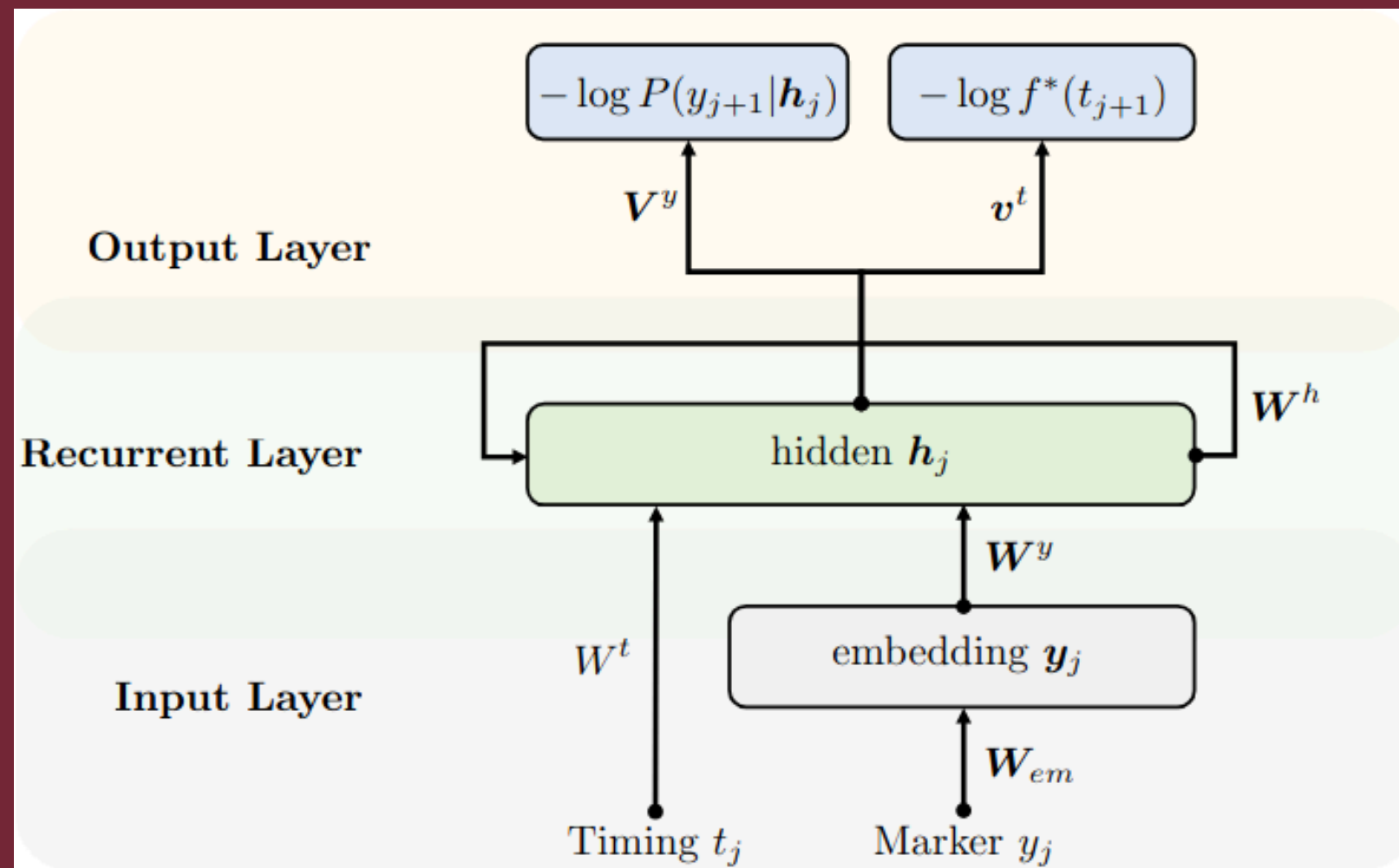


Figure 3: Architect of RMTPP. For a given sequence $\mathcal{S} = ((t_j, y_j)_{j=1}^n)$, at the j -th event, the marker y_j is first embedded into a latent space. Then, the embedded vector and the temporal features are fed into the recurrent layer. The recurrent layer learns a representation that summaries the nonlinear dependency over the previous events. Based on the learned representation h_j , it outputs the prediction for the next marker $\hat{y}_{j+1}$ and timing $\hat{t}_{j+1}$ to calculate the respective loss functions.

Input Layer. At the j -th event, the input layer first projects the sparse one-hot vector representation of the marker y_j into a latent space.

We add an embedding layer with the weight matrix W_{em} to achieve a more compact and efficient representation $y_j = W_{em} y_j + b_{em}$, where b_{em} is the bias. We learn W_{em} and b_{em} while we train the network.

In addition, for the timing input t_j , we can extract the associated temporal features t_j (e.g., like the inter-event duration $d_j = t_j - t_{j-1}$).

Hidden Layer. We update the hidden vector after receiving the current input and the memory h_{j-1} from the past. In RNN, we have

$$\mathbf{h}_j = \max \left\{ \mathbf{W}^y \mathbf{y}_j + \mathbf{W}^t \mathbf{t}_j + \mathbf{W}^h \mathbf{h}_{j-1} + \mathbf{b}_h, 0 \right\}.$$

Marker Generation. Given the learned representation $\mathbf{h}_j$, we model the marker generation with a multinomial distribution by

$$P(y_{j+1} = k | \mathbf{h}_j) = \frac{\exp \left(\mathbf{V}_{k,:}^y \mathbf{h}_j + b_k^y \right)}{\sum_{k=1}^K \exp \left(\mathbf{V}_{k,:}^y \mathbf{h}_j + b_k^y \right)}$$

where K is the number of markers, and $\mathbf{V}_{k,:}^y$ is the k -th row of matrix $\mathbf{V}^y$.

Conditional Intensity. Based on h_j , we can now formulate the conditional intensity function by

$$\lambda^*(t) = \exp \left(\underbrace{\mathbf{v}^{t\top} \cdot \mathbf{h}_j}_{\text{past influence}} + \underbrace{w^t(t - t_j)}_{\text{current influence}} + \underbrace{b^t}_{\text{base intensity}} \right)$$

where $\mathbf{v}^t$ is a column vector, and w^t , b^t are scalars. More specifically,

- The first term $\mathbf{v}^{t\top} \cdot \mathbf{h}_j$ represents the accumulative influence from the marker and the timing information of the past events. Compared to the fixed parametric formulations of (5), (6), and (7) for the past influence, we now have a highly non-linear general specification of the dependency over the history.
- The second term emphasizes the influence of the current event j .
- The last term gives a base intensity level for the occurrence of the next event.
- The exponential function outside acts as a non-linear transformation and guarantees that the intensity is positive.

We can derive the likelihood that the next event will occur at the time t given the history by the following equation:

$$\begin{aligned} f^*(t) &= \lambda^*(t) \exp \left(- \int_{t_j}^t \lambda^*(\tau) d\tau \right) \\ &= \exp \left\{ \mathbf{v}^{t^\top} \cdot \mathbf{h}_j + w^t(t - t_j) + b^t + \frac{1}{w} \exp(\mathbf{v}^{t^\top} \cdot \mathbf{h}_j + b^t) \right. \\ &\quad \left. - \frac{1}{w} \exp(\mathbf{v}^{t^\top} \cdot \mathbf{h}_j + w^t(t - t_j) + b^t) \right\}. \end{aligned} \quad (12)$$

Then, we can estimate the timing for the next event using the expectation

$$\hat{t}_{j+1} = \int_{t_j}^{\infty} t \cdot f^*(t) dt.$$



REMARK



Based on the hidden unit of RNN, we are able to learn a unified representation of the dependency over the history. In consequence, the direct formulation (11) of the conditional intensity function $\lambda \star (t_{j+1})$ captures both of the information from past event timings and event markers.

PARAMETER LEARNING

Given a collection of sequences $\mathcal{C} = \{\mathcal{S}^i\}$, where $\mathcal{S}^i = ((t_j^i, y_j^i)_{j=1}^{n_i})$, we can learn the model by maximizing the joint log-likelihood of observing $\mathcal{C}$.

$$\ell(\{\mathcal{S}^i\}) = \sum_i \sum_j \left(\log P(y_{j+1}^i | \mathbf{h}_j) + \log f(d_{j+1}^i | \mathbf{h}_j) \right), \quad (14)$$

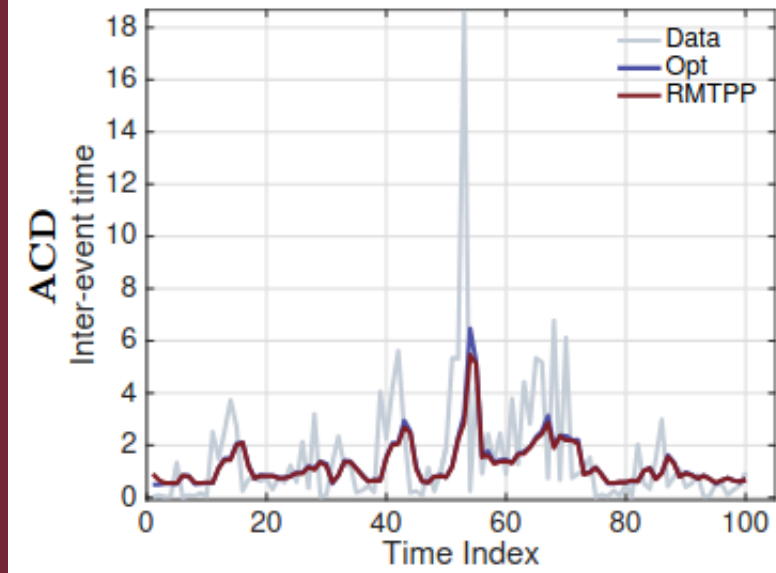
We exploit the Back Propagation Through Time (BPTT) for training RMTTP. Given the size of BPTT as b , we unroll our model in Figure 3 by b steps. In each training iteration, we take b consecutive samples $\{(t_k^i, y_k^i)_{k=j}^{j+b}\}$ from a single sequence, apply the feed-forward operation through the network, and update the parameters with respect to the loss function.

BASELINE

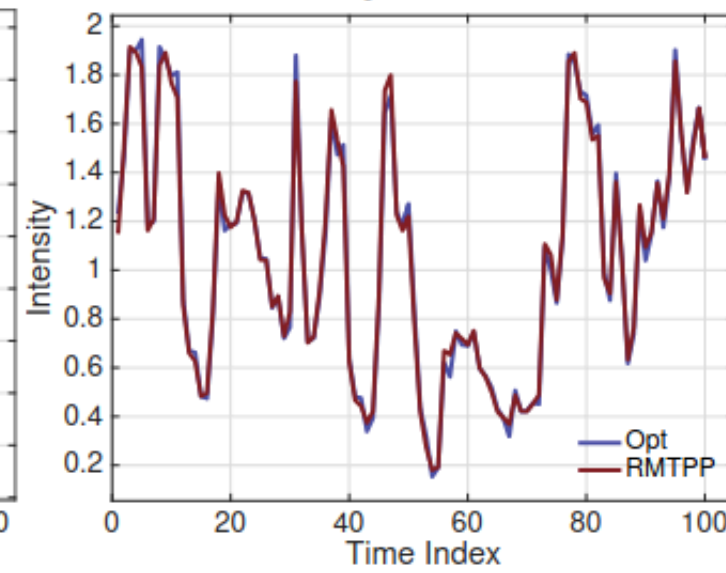
- Majority Prediction. This is also known as the 0-order Markov Chain (MC-0), where at each time step, we always predict the most popular marker regardless of the history. Most often, predicting the most popular type is a strong heuristic.
- Markov Chain. We compare with Markov models with varying orders from one to three, denoted as MC-1, MC-2, and MC-3, respectively. To show the effectiveness of predicting time, we compare with the following continuous-time models:
 - ACD. We fit a second-order autoregressive conditional duration process with the intensity function given in (7).
 - Homogeneous Poisson Process. The intensity function is a constant, which produces an estimate of the average inter-event gaps.
 - Hawkes Process. We fit a self-excitation Hawkes process with the intensity function in (5).
 - Self-correcting Process. We fit a self-correcting process with the intensity function in (6).

Finally, we compare with the Continuous-Time Markov Chain (CTMC) model, which learns continuous transition rates between two states (or markers). This model predicts the next state with the earliest transition time, so it can predict both the marker and the timing for the next event jointly.

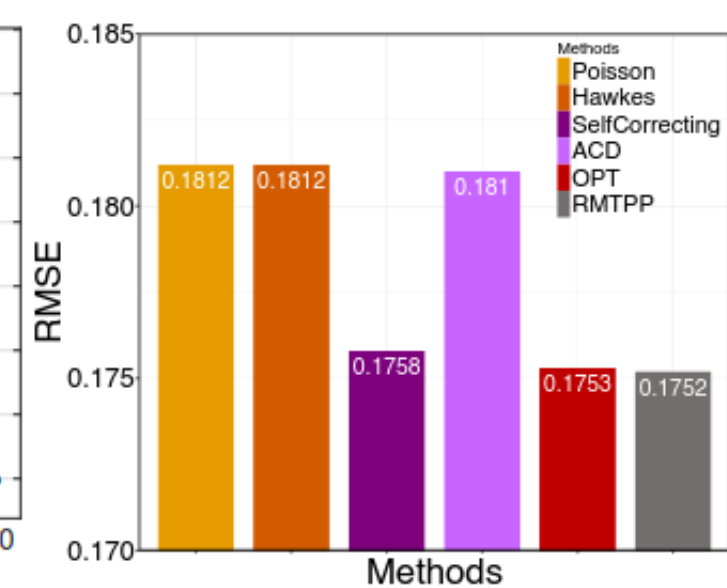
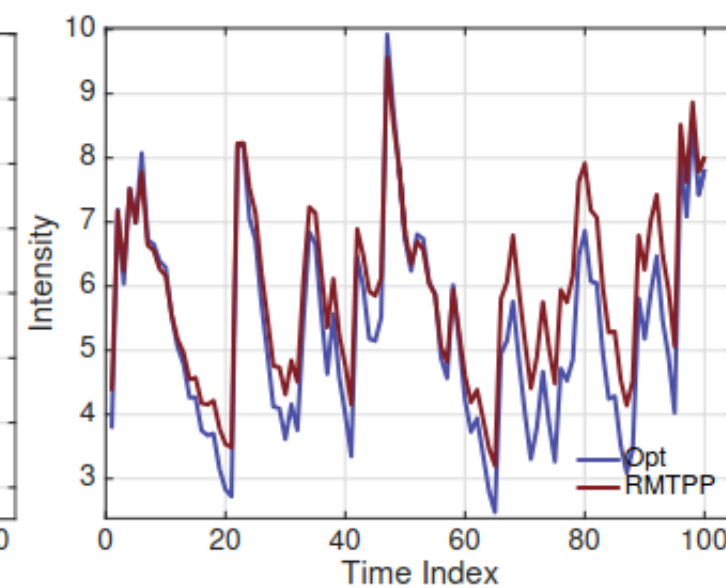
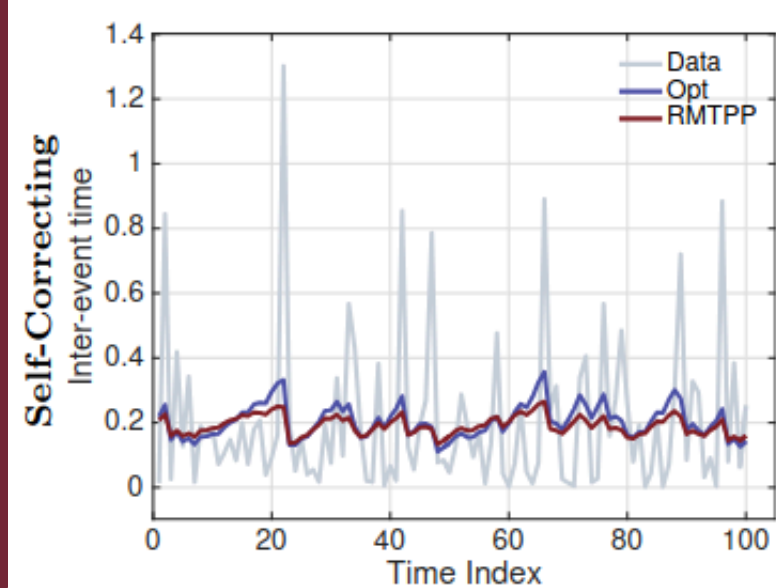
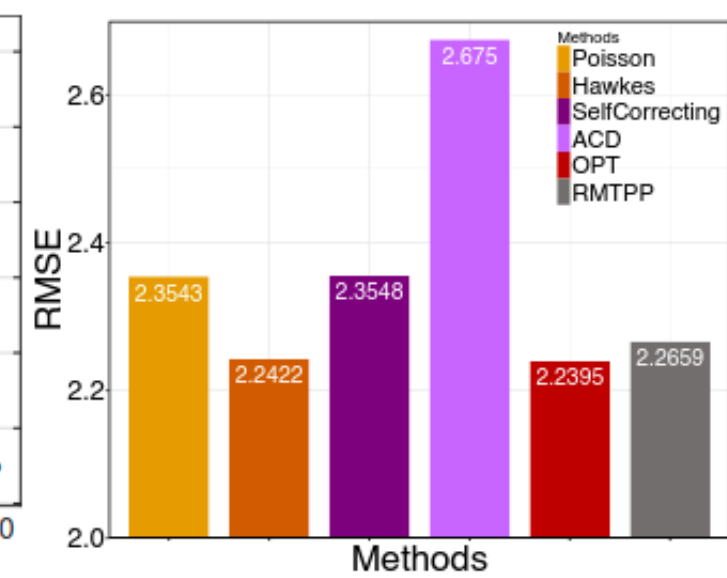
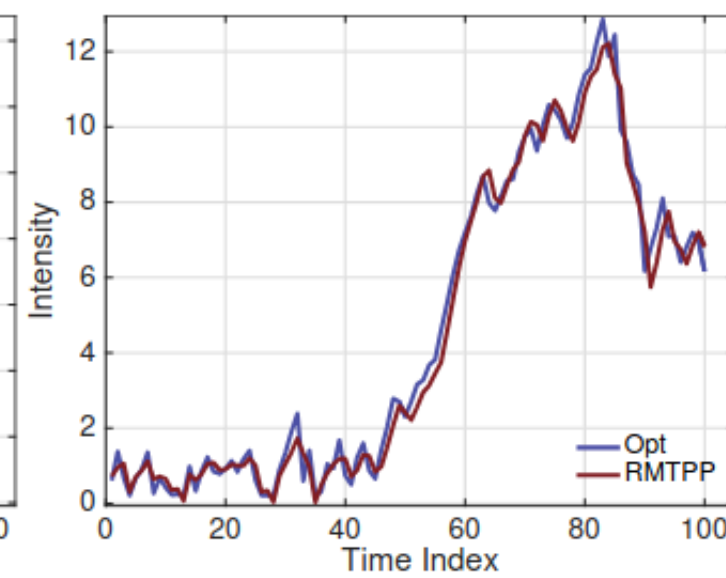
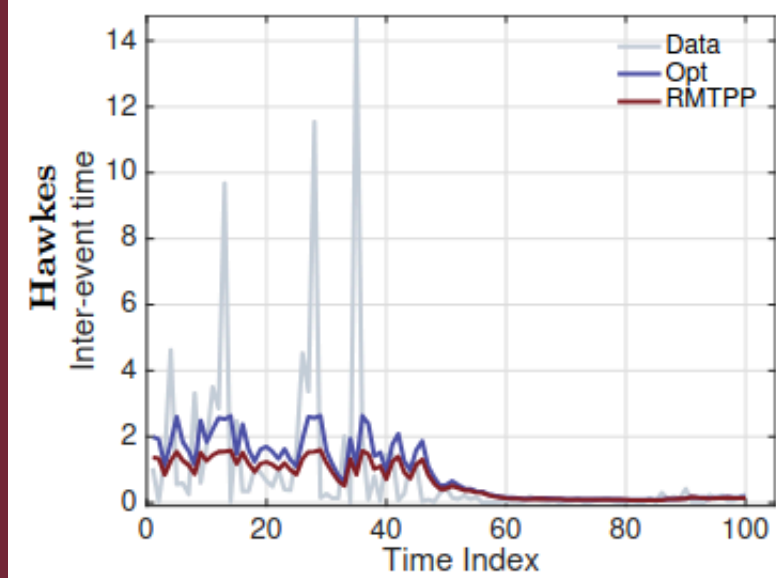
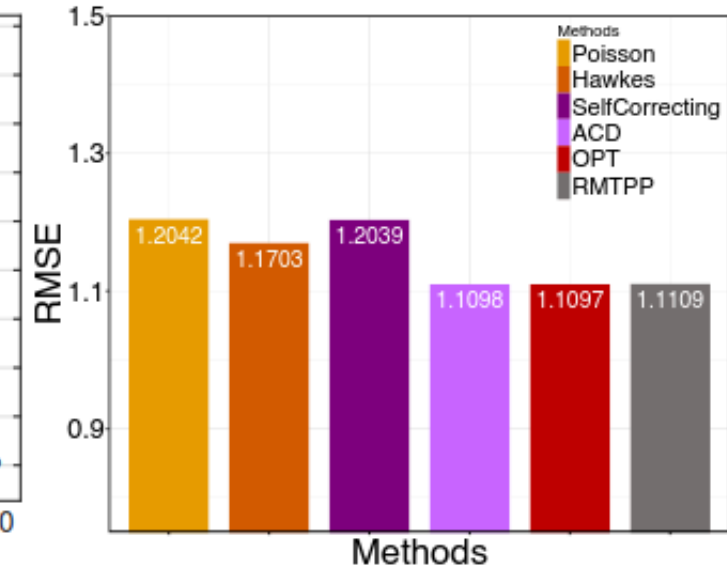
Time Prediction Curve

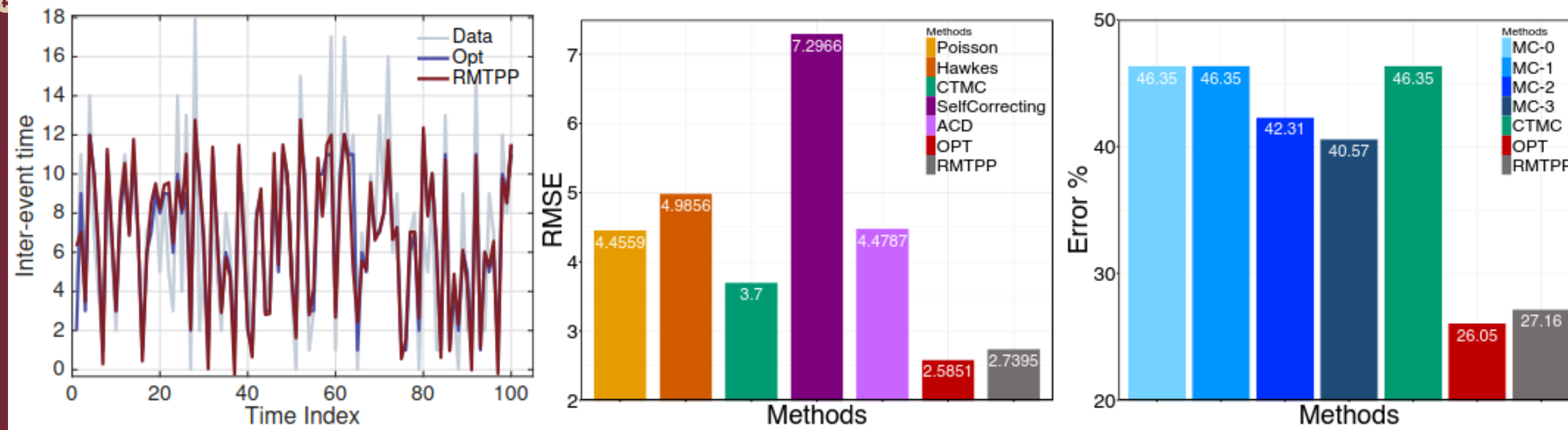


Intensity Function



Time Prediction Error





(a) Time Prediction Curve (b) Time Prediction Error (c) Marker Prediction Error
Figure 5: Performance evaluation of predicting timings and markers on the state-space continuous-time model.

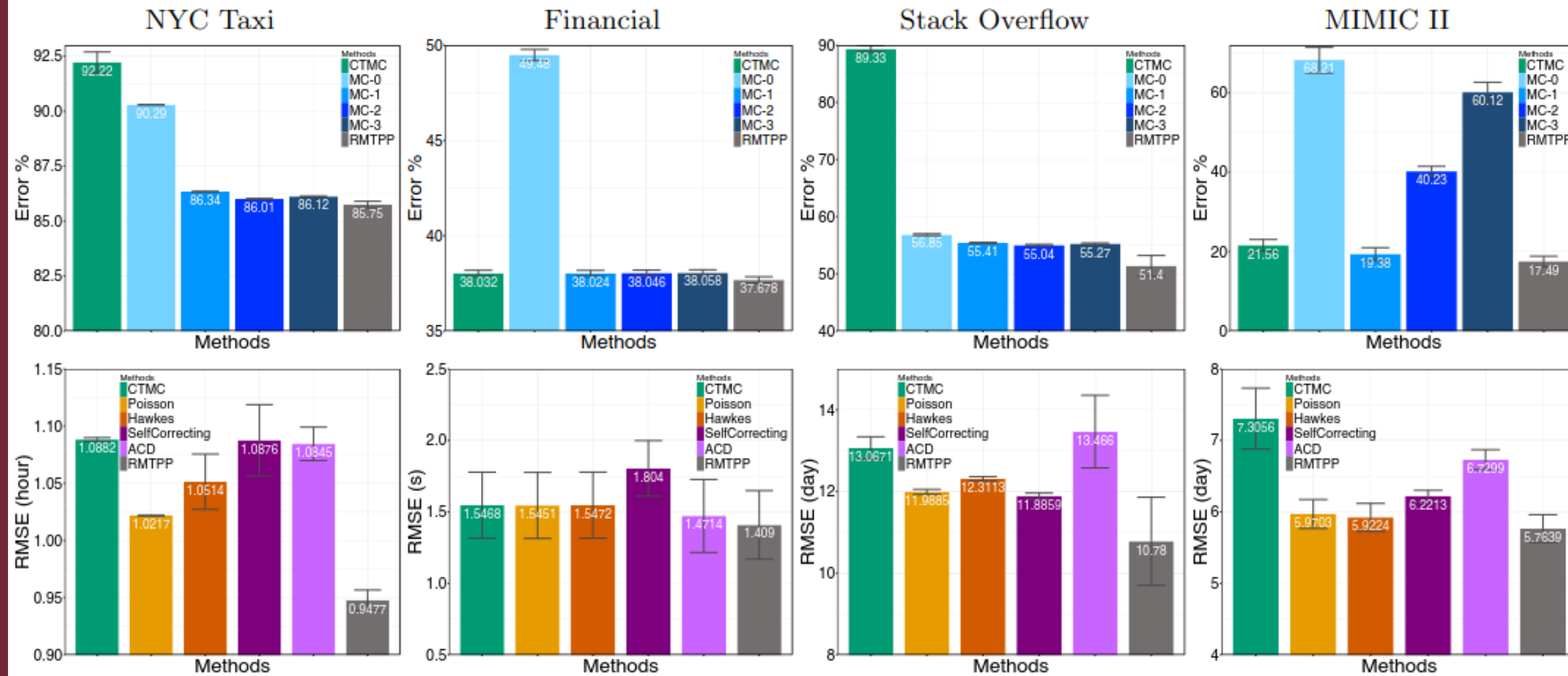
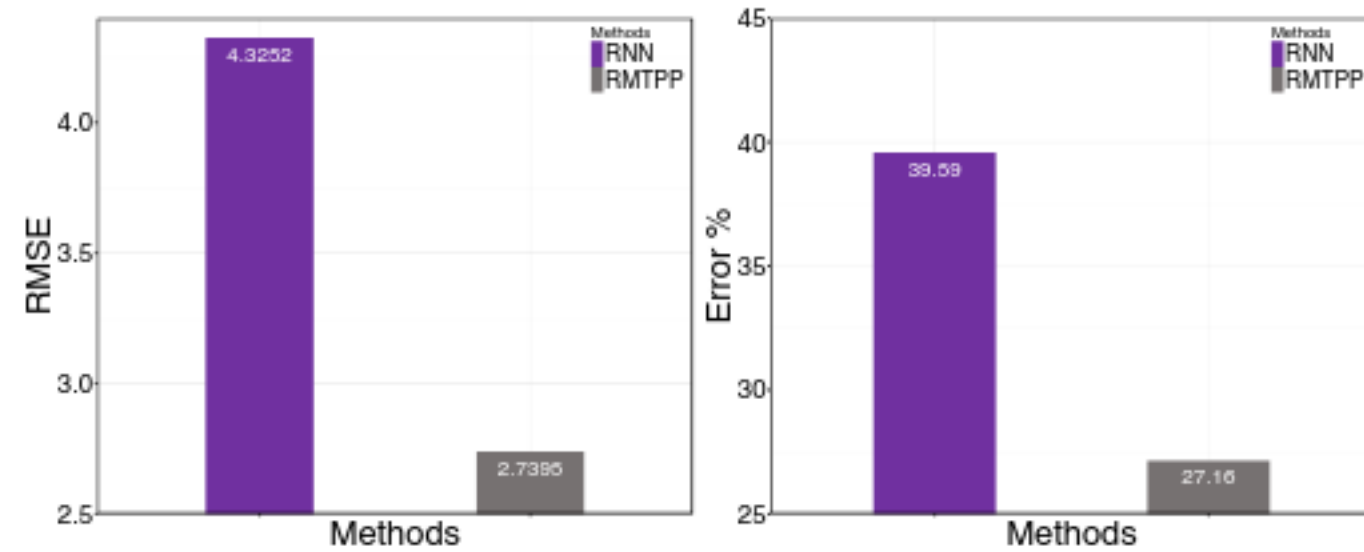
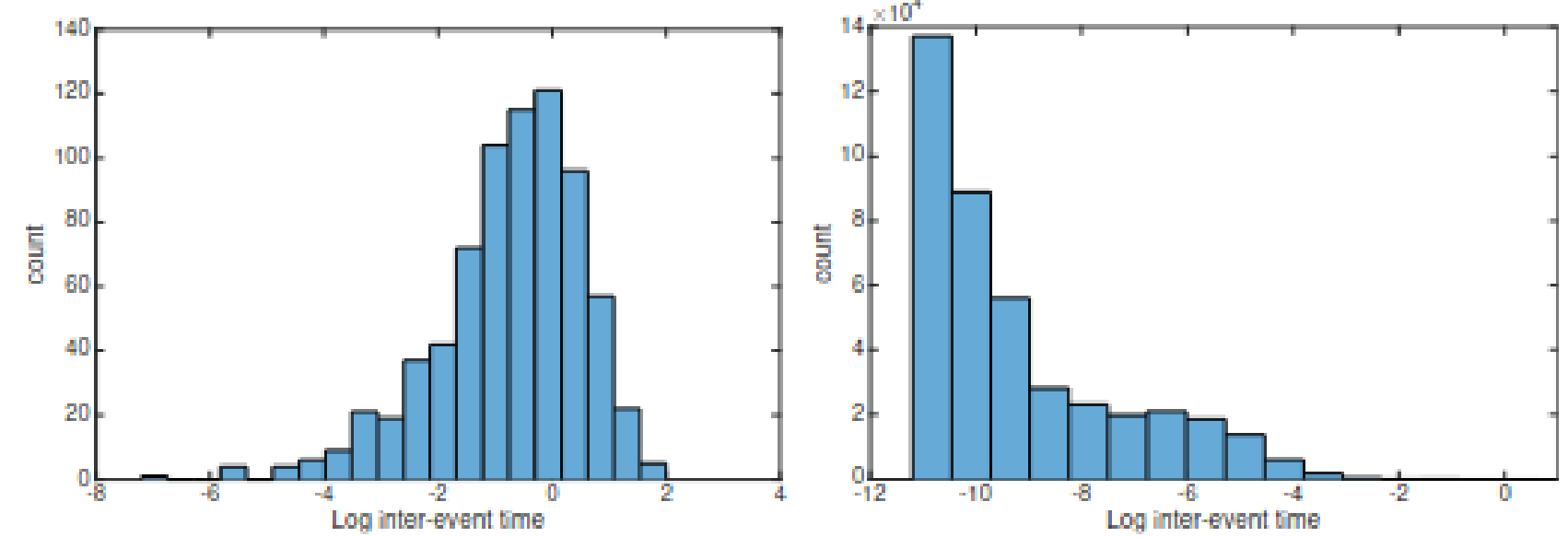


Figure 7: Performance evaluation for predicting both marker and timing of the next event. The top row presents the classification error of predicting the event markers, and the bottom row gives the RMSE of predicting the event timings.



(a) Time Prediction Error (b) Marker Prediction Error
Figure 6: Predictive performance comparison with RNN which is trained for predicting the next timing only in (a), and for predicting the next marker only in (b).



(a) Stack Overflow (b) Financial Transaction
Figure 9: Empirical distribution for the inter-event times. The x-axis is in log-scale.

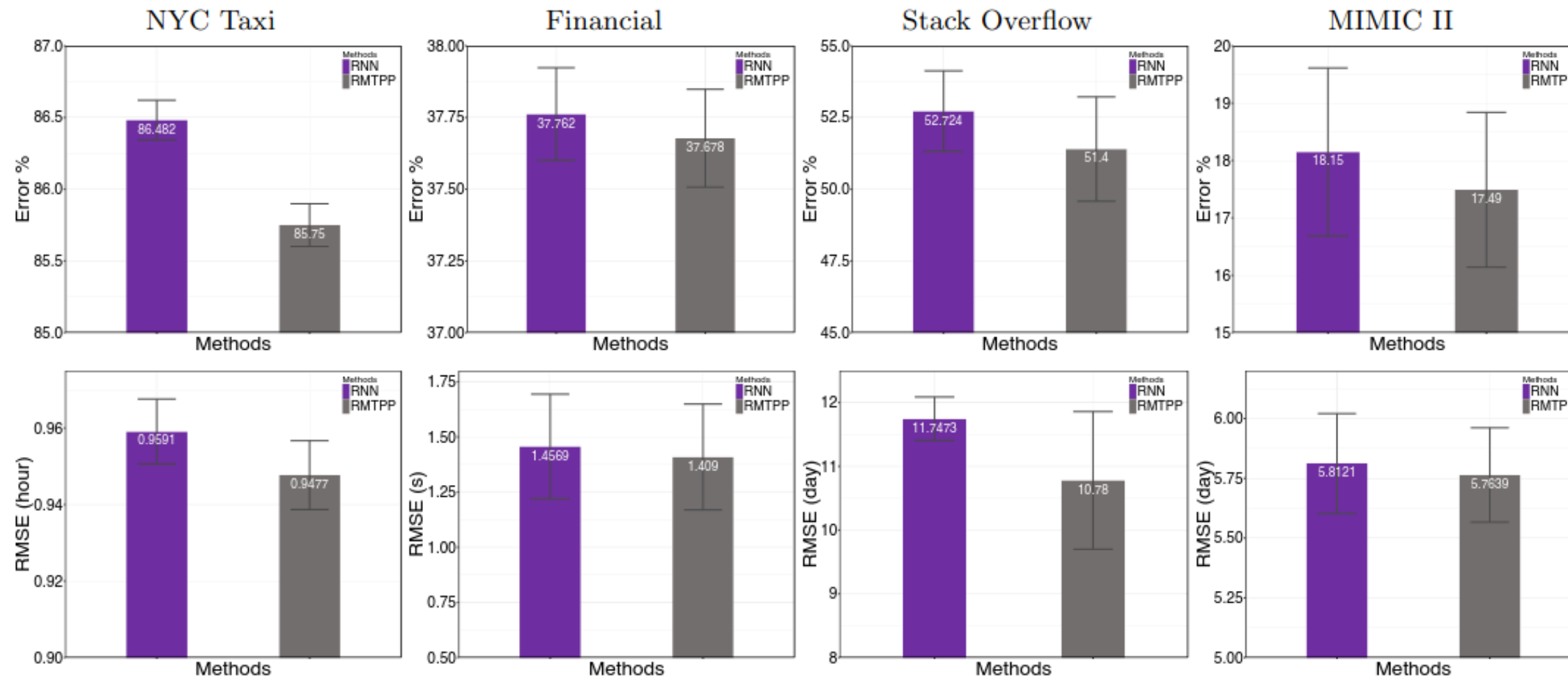


Figure 8: Predictive performance comparison with RNN which are trained only using the markers in the left column and only using the temporal information in the right column.